

Lab 3.1: Network Traffic Prediction

```
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

# -----
# 1. Generate synthetic hourly traffic
# -----
np.random.seed(42)

hours = 24 * 60 # 60 days of hourly data
time_index = pd.date_range("2024-01-01", periods=hours, freq="h")

hour_of_day = time_index.hour
day_of_week = time_index.dayofweek

# Base pattern: morning + evening rush hours, lower at night
daily_pattern = (
    40 * np.exp(-((hour_of_day - 8) ** 2) / 6) + # morning peak
    50 * np.exp(-((hour_of_day - 18) ** 2) / 6) + # evening peak
    10 # baseline
)

# Weekends have lighter traffic
weekend_factor = np.where(day_of_week >= 5, 0.6, 1.0)
```

```

noise = np.random.normal(0, 5, hours)
traffic = daily_pattern * weekend_factor + noise
traffic = np.clip(traffic, 0, None) # no negative traffic

df = pd.DataFrame({"traffic": traffic}, index=time_index)

# -----
# 2. Feature engineering
# -----
df["hour"] = df.index.hour
df["day"] = df.index.dayofweek

# Lag features (previous hours)
for lag in [1, 2, 3, 24]:
    df[f"lag_{lag}"] = df["traffic"].shift(lag)

# Rolling averages
df["roll_mean_3"] = df["traffic"].shift(1).rolling(3).mean()
df["roll_mean_24"] = df["traffic"].shift(1).rolling(24).mean()

df = df.dropna() # drop rows with NaN from lags/rolling

# -----
# 3. Train / test split (time-ordered, no shuffle)
# -----
features = ["hour", "day", "lag_1", "lag_2", "lag_3", "lag_24",
            "roll_mean_3", "roll_mean_24"]

```

```

X = df[features]
y = df["traffic"]

split = int(len(df) * 0.8)
X_train, X_test = X.iloc[:split], X.iloc[split:]
y_train, y_test = y.iloc[:split], y.iloc[split:]

# -----
# 4. Train Random Forest
# -----
model = RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1)
model.fit(X_train, y_train)

# -----
# 5. Evaluate
# -----
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"MSE: {mse:.3f}")
print(f"R2 : {r2:.3f}")

# -----
# 6. Feature importance
# -----

```

```
importance = (pd.Series(model.feature_importances_, index=features)
               .sort_values(ascending=False))
print("\nFeature importance:")
print(importance)
```

Lab 3.2: Network Anomaly Detection

```
import numpy as np
import pandas as pd
from sklearn.ensemble import IsolationForest
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import precision_score, recall_score, f1_score,
classification_report

# -----
# 1. Generate synthetic network data with labels
# -----

np.random.seed(42)

n_normal = 1000
n_anomaly = 50

# Normal traffic
normal = pd.DataFrame({
    "bandwidth": np.random.normal(500, 50, n_normal), # Mbps
    "latency": np.random.normal(20, 5, n_normal), # ms
    "packet_loss": np.random.normal(0.5, 0.2, n_normal), # %
    "connections": np.random.normal(200, 30, n_normal), # active conns
})

# Anomalous traffic (e.g. congestion / attack patterns)
anomaly = pd.DataFrame({
```

```
"bandwidth": np.random.normal(900, 120, n_anomaly), # spikes
"latency": np.random.normal(120, 40, n_anomaly), # high latency
"packet_loss": np.random.normal(8, 3, n_anomaly), # heavy loss
"connections": np.random.normal(600, 150, n_anomaly), # connection floods
})
```

```
df = pd.concat([normal, anomaly], ignore_index=True)
y_true = np.array([0] * n_normal + [1] * n_anomaly) # 1 = anomaly
```

```
# Shuffle so the anomalies aren't all at the end
idx = np.random.permutation(len(df))
df, y_true = df.iloc[idx].reset_index(drop=True), y_true[idx]
```

```
features = ["bandwidth", "latency", "packet_loss", "connections"]
X = df[features]
```

```
# -----
```

```
# 2. Scale features (important for DBSCAN's distance metric)
```

```
# -----
```

```
X_scaled = StandardScaler().fit_transform(X)
```

```
# -----
```

```
# 3. Isolation Forest
```

```
# -----
```

```
contamination = n_anomaly / len(df) # rough prior on anomaly fraction
```

```
iso = IsolationForest(contamination=contamination, random_state=42)
```

```

iso_pred = iso.fit_predict(X_scaled)      # -1 = anomaly, 1 = normal
iso_pred = np.where(iso_pred == -1, 1, 0)  # map to 1 = anomaly

# -----
# 4. DBSCAN
# -----
# Points labeled -1 (noise / not in any cluster) are treated as anomalies
db = DBSCAN(eps=1.0, min_samples=5)
db_labels = db.fit_predict(X_scaled)
db_pred = np.where(db_labels == -1, 1, 0)  # 1 = anomaly

# -----
# 5. Evaluate
# -----
def evaluate(name, y_true, y_pred):
    p = precision_score(y_true, y_pred, zero_division=0)
    r = recall_score(y_true, y_pred, zero_division=0)
    f = f1_score(y_true, y_pred, zero_division=0)
    print(f"\n=== {name} ===")
    print(f"Precision: {p:.3f}")
    print(f"Recall   : {r:.3f}")
    print(f"F1-score  : {f:.3f}")

evaluate("Isolation Forest", y_true, iso_pred)
evaluate("DBSCAN", y_true, db_pred)

```

Lab 3.3: Customer Segmentation

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

# -----
# 1. Generate synthetic customer data (a few natural groups)
# -----

np.random.seed(42)

def make_segment(n, data, minutes, sms, revenue, spread):
    return pd.DataFrame({
        "data_usage": np.random.normal(data, spread[0], n), # GB / month
        "minutes": np.random.normal(minutes, spread[1], n), # call minutes
        "sms": np.random.normal(sms, spread[2], n), # texts
        "revenue": np.random.normal(revenue, spread[3], n), # $ / month
    })

# Heavy data users, talkers, texters, low-usage
df = pd.concat([
    make_segment(300, 25, 200, 50, 70, (5, 40, 15, 10)),
    make_segment(300, 5, 900, 100, 60, (2, 100, 30, 8)),
    make_segment(300, 8, 150, 600, 45, (3, 40, 80, 7)),
```



```

        make_segment(300, 2, 100, 40, 20, (1, 30, 15, 5)),
], ignore_index=True)

df = df.clip(lower=0) # no negative values

features = ["data_usage", "minutes", "sms", "revenue"]
X = df[features]

# -----
# 2. Scale (K-means uses Euclidean distance)
# -----
X_scaled = StandardScaler().fit_transform(X)

# -----
# 3. Choose k using silhouette score
# -----
scores = {}
for k in range(2, 8):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    scores[k] = silhouette_score(X_scaled, labels)

best_k = max(scores, key=scores.get)
print("Silhouette score by k:")
for k, s in scores.items():
    print(f" k={k}: {s:.3f}")
print(f"\nBest k = {best_k} (silhouette = {scores[best_k]:.3f})")

```

```

# -----
# 4. Final K-means with best k
# -----

kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
df["cluster"] = kmeans.fit_predict(X_scaled)

# Segment profiles (average feature values per cluster)
print("\nSegment profiles:")
print(df.groupby("cluster")[features].mean().round(1))

# -----
# 5. Visualize
# -----

# (a) Silhouette vs k
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].plot(list(scores.keys()), list(scores.values()), "o-")
axes[0].axvline(best_k, color="red", linestyle="--", label=f"best k={best_k}")
axes[0].set_xlabel("Number of clusters (k)")
axes[0].set_ylabel("Silhouette score")
axes[0].set_title("Choosing k")
axes[0].legend()

# (b) Clusters projected to 2D with PCA
coords = PCA(n_components=2).fit_transform(X_scaled)
scatter = axes[1].scatter(coords[:, 0], coords[:, 1],

```

```
c=df["cluster"], cmap="tab10", s=15, alpha=0.7)
axes[1].set_xlabel("PCA component 1")
axes[1].set_ylabel("PCA component 2")
axes[1].set_title("Customer segments (PCA view)")
plt.colorbar(scatter, ax=axes[1], label="cluster")

plt.tight_layout()
plt.savefig("customer_segments.png", dpi=120)
plt.show()
```